Лабораторная работа №3

Оформление отчётов с помощью Markdown

Мацюк Константин Владимирович

2026

Российский университет дружбы народов

Цель работы

Задание

Введение в Markdown

Выполнение лабораторной работы

Выводы

Список литературы

Цель работы

Цель работы

Научиться оформлять отчёты с помощью легковесного языка разметки Markdown.

Задание

Задание

- Сделать отчёт по предыдущей лабораторной работе в формате Markdown
- Предоставить файлы в 3 форматах: pdf, docx и md (в архиве, включая скриншоты и Makefile)

Введение в Markdown

Введение в Markdown

Что такое Markdown?

Markdown — это облегчённый язык разметки, разработанный для форматирования текста с сохранением его читаемости в исходном виде.

Синтаксические элементы Markdown

Элемент	Синтаксис	Назначение
Заголовки	`# Текст`	Уровни заголовков от 1 до 6
Полужирный	`**текст**`	Выделение текста полужирным наче
Курсив	`*текст*`	Выделение текста курсивом
Маркированный список	`- элемент`	Создание списка с маркерами
Нумерованный список	`1. элемент`	Создание упорядоченного списка
Гиперссылки	`[текст](адрес)`	Вставка ссылок на внешние ресурсы
Изображения	`![подпись](путь)`	Встраивание графических файлов

Синтаксические элементы Markdown

Элемент	Синтаксис	Назначение
Встроенный код	`` `код` ``	Выделение фрагментов программного кода
Блоки кода	```` ````	Оформление многострочных листингов
Математические формулы	$выражение$	Вставка формул в формате LaTeX

Инструмент Pandoc

Для конвертации Markdown-документов в целевые форматы применяется инструмент **Pandoc** (https://pandoc.org/).

Базовые команды:

```
pandoc README.md -o README.pdf   # генерация PDF
pandoc README.md -o README.docx  # генерация DOCX
```

Автоматизация сборки реализуется с помощью **Makefile**.

Выполнение лабораторной работы

Заполнение YAML-шапки

Открываю файл report.md в текстовом редакторе. Вношу данные в YAML-шапку: название работы, дисциплину, сведения об авторе и параметры форматирования.

```yaml
---
## Front matter
title: "Лабораторная работа №2. Первоначальная настройка git"
subtitle: "Дисциплина: Архитектура компьютеров и операционные системы"
author: "Мащек Константин Владимирович"

## Generic otions
lang: ru-RU
toc-title: "Содержание"

## Bibliography
bibliography: bib/cite.bib
csl: pandoc/csl/gost-r-7-0-5-2008-numeric.csl

## Pdf output format
toc: true
toc-depth: 2
lof: true
lot: true
fontsize: 12pt
linestretch: 1.5
papersize: a4
documentclass: scrreprt
## I18n polyglossia
polyglossia-lang:
  name: russian
  options:
    - spelling=modern
    - babelshorthands=true
polyglossia-otherlangs:
  name: english
## I18n babel
babel-lang: russian
babel-otherlanos: enolish
```

Рис. 1: Заполнение YAML-шапки документа

Оформление цели и задания

Формирую цель работы и задание.

```
# Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

# Задание

- Создать базовую конфигурацию для работы с git
- Создать ключ SSH
- Создать ключ PGP
- Настроить подписи git
- Зарегистрироваться на Github
- Создать локальный каталог для выполнения заданий по предмету
```

Рис. 2: Оформление цели и задания работы

Оформление хода работы

Описываю процесс выполнения лабораторной работы №2, добавляя текстовые описания действий, блоки кода с командами и ссылки на скриншоты.

```
# Выполнение лабораторной работы

## Устанавливаю программного обеспечения

Устанавливаю git и gh с помощью dnf (рис. -@fig:001).

![Установка git и gh]({image/1.png}){#fig:001 width=70%}

## Базовая настройка git

Выполняю базовую настройку git: задаю имя и email владельца репозитория, настраиваю utf-8 в выводе сообщений, задаю имя начальной ветки master, настраи
параметры autocrlf и safecrlf (рис. -@fig:002).

![Базовая настройка git]({image/2.png}){#fig:002 width=70%}

## Создание ключей SSH

Создаю SSH ключ по алгоритму RSA с размером 4096 бит (рис. -@fig:003).

![Генерация RSA ключа]({image/3.png}){#fig:003 width=70%}

## Добавление SSH ключа на GitHub

Перехожу в настройки GitHub, добавляю ключ с названием Lab (рис. -@fig:004).

![Добавление SSH ключа на GitHub]({image/4.png}){#fig:004 width=70%}

## Создание ключа PGP

Генерирую PGP ключ командой gpg --full-generate-key. Выбираю тип RSA and RSA, размер 4096 бит, срок действия 0. Вводу имя и email (рис. -@fig:005).

![Генерация PGP ключа]({image/5.png}){#fig:005 width=70%}

Просматриваю список ключей и копирую fingerprint ключа (рис. -@fig:006).

![Просмотр списка PGP ключей]({image/6.png}){#fig:006 width=70%}

Экспортирую ключ для добавления на GitHub и копирую в буфер обмена (рис. -@fig:007).

![Экспорт PGP ключа]({image/7.png}){#fig:007 width=70%}
```

Рис. 3: Оформление хода работы

Оформление ответов на контрольные вопросы

Создаю раздел с ответами на контрольные вопросы. Применяю нумерованные списки, выделение ключевых понятий и блоки с примерами команд.

```markdown
# Ответы на контрольные вопросы

**1. Что такое системы контроля версий (VCS)?**

VCS — это программное обеспечение, предназначенное для регистрации изменений,
вносимых в файлы проекта. Оно даёт возможность сохранять состояния файлов,
восстанавливать более ранние версии, а также организовать совместную работу
нескольких специалистов над общей кодовой базой, минимизируя конфликты.

**2. Хранилище, коммит, история, рабочая копия:**

- **Хранилище (репозиторий)** - хранилище всех версии файлов
  и журнал всех изменений проекта.
- **Коммит** — это зафиксированное состояние файлов проекта на конкретный
  момент, сопровождаемое комментарием о проделанных изменениях.
- **История** — это хронологический список всех коммитов, содержащий информацию
  об авторе, времени и комментарии к каждому изменение.
- **Рабочая копия** — это актуальное содержимое файлов проекта, доступное
  разработчику на его локальном компьютере для редактирования.

**3. Централизованные и распределённые VCS:**

- **Централизованные** — предполагает наличие единственного сервера с полной
  историей версий, а разработчики получает только рабочие копии с этого сервера.
  К таким системам относятся CVS и Subversion (SVN).
- **Распределённые** — предоставляет каждому участнику полную локальную копию
  репозитория, включая всю историю. Примерами являются Git и Mercurial.

**4. Единоличная работа с хранилищем**

- Инициализация локального хранилища
- Добавление файлов под контроль версий
- Фиксация изменений с описанием
- Просмотр истории изменений
- Возврат к предыдущим версиям при необходимости

**5. Работа с общим хранилищем**

- Получение актуальной версии проекта из общего хранилища
- Внесение изменений в локальной рабочей копии
- Фиксация изменений в локальном репозитории
- Отправка зафиксированных изменений в общее хранилище
- Разрешение конфликтов при одновременном изменении одних файлов
```

Рис. 4: Оформление контрольных вопросов

Оформление ответов на контрольные вопросы

```
**6. Основные функции Git:**

Ключевые возможности Git включают: контроль изменений в файлах, ведение
подробной истории версий, создание и управление ветками, объединение изменений
от нескольких участников и возможность отката к предыдущим состояниям проекта.

**7. Команды Git:**

Основные команды Git:

| Команда | Описание |
|---------|----------|
| `git init` | Создание нового репозитория |
| `git clone` | Клонирование удалённого репозитория |
| `git add` | Добавление файлов в индекс |
| `git commit` | Фиксация изменений |
| `git push` | Отправка изменений на сервер |
| `git pull` | Получение изменений с сервера |
| `git status` | Просмотр состояния рабочей копии |
| `git log` | Просмотр истории коммитов |
| `git branch` | Управление ветками |
| `git merge` | Слияние веток |

**8. Примеры действий с репозиторием:**

Локальный репозиторий:

```bash
git init
echo "hello" > file.txt
git add file.txt
git commit -m "начальный коммит"
```
```

Рис. 5: Оформление таблицы

Оформление выводов

Завершаю отчёт разделом с выводами.

```
# Выводы

В ходе выполнения лабораторной работы изучил идеологию и применение средств контроля версий. Освоил базовую настройку git, создание SSH и PGP ключей, настройку
подписей коммитов. Настроил GitHub CLI (gh) для удобной работы с GitHub из командной строки. Создал репозиторий курса на основе шаблона и настроил его для
дальнейшей работы.
```

Рис. 6: Оформление выводов

Компиляция отчёта

Компилирую отчёт с помощью команды make. Pandoc обрабатывает файл report.md и генерирует документы в форматах PDF и DOCX.

```
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$ make
pandoc "report.md" --filter pandoc-crossref --number-sections --citeproc -o "report.docx"
pandoc "report.md" --filter pandoc-crossref --pdf-engine=lualatex --pdf-engine-opts=--shell-escape  --citeproc  --number-sections -o "report.pdf"
```

Рис. 7: Запуск команды make для компиляции отчёта

Проверка результатов

Проверяю содержимое каталога командой ls. Убеждаюсь в наличии сгенерированных версий report.pdf и report.docx.

```
kvmacyuk@kvmacyuk:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab02/report$ ls
bib  image  Makefile  pandoc  report.docx  report.md  report.pdf
```

Рис. 8: Проверка содержимого каталога

Выводы

Выводы

В ходе выполнения лабораторной работы:

- Изучены основные элементы разметки Markdown (заголовки, списки, таблицы)
- Освоена конвертация документов в PDF и DOCX с помощью Pandoc
- Настроена автоматизация сборки через Makefile

Список литературы